

Go で作るリンクチェッカーの仕組み

著者：関 勝寿 公開日：2025 年 11 月 19 日 ソース：<https://sekika.github.io/2025/11/19/go-linkchecker/>

Web サイトのリンクチェックを自動化するには、多数の URL を効率よく処理しつつ、アクセス先に負荷をかけない工夫が必要になります。Go は goroutine とチャンネルを用いた並列処理を得意としており、この仕組みを使うことでリンクチェックも高速化できます。

しかし、単に並列化すると同じホストに短時間で大量アクセスしてしまい、相手サーバーに迷惑をかける可能性があります。そこで今回紹介するリンクチェッカーのコードでは、ホストごとに専用のワーカー（goroutine）を割り当て、同じホストには一定間隔を空けてアクセスするという仕組みを実装しています。

この記事では、このアクセス制御をどのように実現しているのか、Go 製リンクチェッカーの FetchHTTP と RunWorkers の 2 つの関数を解説します。最新版のコードとは若干異なります。

!Go Reference](<https://pkg.go.dev/badge/github.com/sekika/linkchecker.svg>)

1.FetchHTTP：実際に GET してステータスを調べる関数

```
func FetchHTTP(link string, client *http.Client, userAgent string) error {  
    req, err := http.NewRequest("GET", link, nil)  
    if err != nil {  
        return err  
    }  
    req.Header.Set("User-Agent", userAgent)  
    req.Header.Set("Accept",  
        "text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8")  
    req.Header.Set("Accept-Language", "en-US,en;q=0.5")  
    req.Header.Set("Accept-Encoding", "gzip, deflate")  
    req.Header.Set("Connection", "keep-alive")  
    req.Header.Set("Cache-Control", "max-age=0")  
    req.Header.Set("Sec-Fetch-Dest", "document")  
    resp, err := client.Do(req)  
  
    if err != nil {  
        return err  
    }  
    defer resp.Body.Close()  
    if resp.StatusCode >= 400 {  
        return fmt.Errorf("status %d", resp.StatusCode)  
    }  
    return nil  
}
```

- GET を送る
- User-Agent を設定
- ステータスコードでエラーを返す

というシンプルな HTTP アクセス関数です。

2.RunWorkers：ホストごとにワーカーを作り、リンクを処理する

ここからが全体の中心になる RunWorkers 関数です。

```
func RunWorkers(  

```

```
    links []string,  
    baseURL string,  
    timeoutSec int,  
    waitSec int,  
    userAgent string,  
) {  

```

2.1.1. リンクの変換

まずは relative URL の解決を行います。ここで、除外するホストを指定するといった処理を入れることも可能です。

```
base, _ := url.Parse(baseURL)  
filteredLinks := []string{  
  
    for _, link := range links {  
        u, err := base.Parse(link)  
        if err != nil {  
            continue  
        }  
  
        filteredLinks = append(filteredLinks, u.String())  
    }  

```

2.2.2. ホストごとのチャンネルを保持する map を用意

```
hostQueues := make(map[string]chan string)  
var mu sync.Mutex  
var wg sync.WaitGroup
```

ここでは、ホストごとの専用キュー（channel）を入れていく map を用意しています。

• mu は map へのアクセスを安全にするための排他制御用の Mutex です。複数の goroutine が同時に map を読み書きしても競合が起きないようにします。

• wg は起動中のワーカーの数を管理し、すべてのワーカーが終了するまで待つための WaitGroup です。各ワーカーの起動時にカウンターを増やし、終了時に減らすことで、最後に wg.Wait() で全てのワーカーが処理を終えるまで待つことができます。

2.3.3. リンクを 1 件ずつ処理し、ホストごとにワーカーを作る

重要なループがこれです。

```
for _, link := range filteredLinks {  
    u, _ := url.Parse(link)  
    host := u.Host
```

この for 文は、

- すべてのリンクを 1 行ずつ取り出す
- そのリンクのホスト名（例：example.com）を取り出す

という処理を行っています。

このあとに、そのホスト専用のチャンネルとワーカーが必要かどうか判定する処理が続きます。

2.3.1.3-1. ホスト用のキューが無いなら作る

```
    mu.Lock()  
    if _, ok := hostQueues[host]; !ok {
```

```
hostQueues[host] = make(chan string, 100)
ch := hostQueues[host]
```

この if 文は for の中にあり、つまりリンクごとに毎回「このホストのキューは作り済みか？」をチェックしていることになります。

新しいホストなら：

- ・新しいチャンネルを作成
- ・そのチャンネルを処理するワーカー goroutine を起動

という流れになります。

2.3.2.3-2. ワーカー goroutine の起動（ホストごとに1つだけ）

```
wg.Add(1)
go func(host string, ch chan string) {
    defer wg.Done()
    jar, _ := cookiejar.New(nil)
    client := &http.Client{
        Timeout: time.Duration(timeoutSec) * time.Second,
        Jar: jar,
    }
    for l := range ch {
        err := FetchHTTP(l, client, userAgent)
        if err != nil {
            fmt.Printf("[NG] %s (%v)\n", l, err)
        } else {
            fmt.Printf("[OK] %s\n", l)
        }
        time.Sleep(time.Duration(waitSec) * time.Second)
    }
}(host, ch)
}
```

この goroutine は、そのホストに属するリンクだけを順番に処理します。

for l := range ch により、チャンネルが閉じられるまで動作します。

そして、各アクセスの後には必ず

```
time.Sleep(waitSec)
```

を入れることで、同一ホストへのアクセス間隔があくようになっています。

このワーカー内部では、まず client := &http.Client{Timeout: ..., Jar: jar} により、各ホスト専用の HTTP クライアントが生成されており、アクセスごとにタイムアウト時間を設定することで、応答が返ってこないリンクに対して無限に待ち続けないようにしています。ここで cookiejar をつけておきます。

2.3.3.3-3. 作成済みの（または前に作った）ホスト用チャンネルにリンクを送る

```
hostQueues[host] <- link
}
```

この部分も for の一部で、「今扱っているリンクを、対応するホストのチャンネルへ入れる」という動作になっています。

2.4.4. 全チャンネルを閉じる

```
for _, ch := range hostQueues {
    close(ch)
}
```

チャンネルを閉じることで、ワーカー側の

```
for l := range ch
```

が終わり、各 goroutine が終了します。

2.5.5. ワーカーの終了を待つ

```
wg.Wait()
```

リンクチェッカーでは、ホストごとに goroutine を起動してリンクを並列処理しています。しかし、プログラムの最後ですべてのワーカーが処理を終えるのを待たずに終了してしまうと、まだチェック中のリンクが途中で止まってしまう可能性があります。ここで登場するのが var wg sync.WaitGroup で定義された wg です。

wg は複数の goroutine の終了を待つのためのカウンターです。このプログラムでは以下のように使われています。

1. ワーカー起動前にカウンターを増やす (wg.Add(1))：ホストごとのワーカー goroutine を作る直前で wg.Add(1) が呼ばれています。これで「この goroutine の終了を待つ」という状態になります。
2. ワーカー終了時にカウンターを減らす (wg.Done())：ワーカー内部の最初で defer wg.Done() が置かれているため、goroutine が処理を終えた時に自動的にカウンターが減ります。
3. 全ワーカーの終了を待つ (wg.Wait())：最後にメインルーチンで wg.Wait() を呼ぶことで、すべてのホスト用ワーカーがリンクチェックを完了するまでプログラムが終了しないようにしています。

この仕組みにより、複数のホストを並列に処理しつつも、すべてのリンクチェックが完了してからプログラムを終了させることができます。

3. 全体構造をまとめると

1. まず URL リストをフィルタする
2. for _, link := range filteredLinks のループが始まる
3. ループの中で
 - ・ホスト名を取得
 - ・そのホストのキューが map に無ければ作る
 - ・対応するチャンネルにリンクを送る
4. ループがすべて終わったらチャンネルを閉じる
5. ワーカーが終わるまで待つ

という流れになっています。

4. 必要な import とその使用箇所

このリンクチェッカーでは、標準ライブラリを中心にいくつかのパッケージを利用しています。

```
import (
    "fmt"
    "log"
    "net/http"
    "net/http/cookiejar"
    "net/url"
```

```
    ⓧ "sync"  
    ⓧ "time"  
)
```

主な import とその役割は次のとおりです。net/http はリンク先にアクセスするための HTTP クライアントで使用され、net/http/cookiejar は cookiejar で使用され、net/url は baseURL と相対リンクを結合して正規化する処理に使われています。time はアクセス間隔の調整やタイムアウト設定に使われ、log は各リンクのチェック結果を表示するために使われています。sync はホストごとのワーカーを安全に管理するための Mutex と WaitGroup に利用され、fmt は HTTP エラーの整形表示に使われます。

5. おわりに

以上は、AI が書いたプログラムを AI に解説させたものです。AI にプログラムを書かせて動くことを確認し（期待通りに動くまで何度も書き直させて）、その解説文を AI に書かせて（分かりにくいところは何度か書き直させて）それをブログに掲載し、それを読んで自分が理解するという AI に依存したプログラミング言語の学習法です。このプログラムは[授業で使っているリンク集](#)のリンク切れをチェックするために、実際に使っています。